

Submission Date: 13th May 2026

Create a single zip file with all the files you want to submit. Name the file with the surname of all members of your group (e.g., `x_y_z.zip`). Upload your zip in the “Submission” folder of the corresponding week (Exercises → Week 3 - Transformers-Part-2 → Submission).

1 Comparing ViT and CNN Architectures

While Convolutional Neural Networks (CNNs) have historically dominated computer vision due to their architectural alignment with image properties, Vision Transformers (ViTs) introduce a significant paradigm shift that impacts how the models learn, as well as their efficiency & data requirements.

- Input Processing:** The Vision Transformer (ViT) introduces a radically different image encoding strategy compared to convolutional networks. First, explain the high-level steps ViT uses to prepare an image for its Transformer Encoder. Next, analyze how this process differs fundamentally from the hierarchical, local feature extraction performed by a standard CNN on the same input. What are the implications of these differing approaches for model inductive bias?
- Data Efficiency:** Explain why ViTs typically require significantly larger datasets to train from scratch compared to CNNs to achieve comparable performance. (*hint*: what specific relationships must the ViT learn from scratch that the CNN has hard-coded?)
- Complexity Analysis:** For a high-resolution image with N patches, define the computational complexity (using big O notation) of the global self-attention mechanism in ViT. Explain why this creates a bottleneck for processing high-resolution images compared to the local operations of a CNN.

2 Swin Transformer: Architecture & Computational Cost

Context: You are designing a Swin Transformer backbone for a segmentation model processing high-resolution aerial surveillance imagery. The input images are tiles of resolution $H \times W = 2048 \times 2048$ pixels. To ensure real-time latency on edge hardware, you must precisely calculate the tensor dimensions and floating-point operations (FLOPs) at specific stages.

Model Configuration:

- Input:** $H \times W \times C = 2048 \times 2048 \times 3$ (RGB)
- Patch Size:** $P = 4$
- Window Size:** $M = 8$ (height/width of local windows in patches)
- Base Channel Dimension:** $C = 96$ (Output of Stage 1 Linear Embedding)
- Patch Merging Ratio:** The linear projection reduces the concatenated channel dimension by a factor of 2

Tasks:

- Tensor Dimensionality Tracking:** Trace the exact dimensions of the feature maps through the network hierarchy.
 - Stage 1 Input:** Calculate the tensor shape ($H_1 \times W_1 \times C_1$) at the output of the **Linear Embedding** layer (which serves as the input to the Swin Transformer Blocks in Stage 1).
 - Patch Merging (Stage 1 → Stage 2):** The transition to Stage 2 involves a Patch Merging layer. This layer concatenates features from 2×2 neighboring patches and applies a linear projection.
 - Calculate the intermediate channel dimension C_{inter} *after* concatenation but *before* the linear projection.
 - Calculate the final tensor shape ($H_2 \times W_2 \times C_2$) entering Stage 2.
 - Stage 3 Input:** Calculate the tensor shape ($H_3 \times W_3 \times C_3$) entering Stage 3 after the second Patch Merging operation.

- b) **Window Partitioning (Stage 2):** In Stage 2, the network partitions the feature map into non-overlapping local windows for the W-MSA (Window-based Multi-Head Self-Attention) mechanism.
- Using the dimensions ($H_2 \times W_2$) derived above, calculate the total number of windows $N_{windows}$ in this stage.
 - How many feature patches ($N_{patches_per_window}$) are contained within a single window?
- c) **FLOPs Analysis (The Quadratic Bottleneck):** A standard Global Self-Attention (MSA) mechanism has a computational complexity dominated by the attention matrix calculation:

$$\text{FLOPs}^{MSA} \approx 4hwC^2 + 2(hw)^2C \quad (1)$$

In contrast, the Window-based Self-Attention (W-MSA) restricts this to local windows:

$$\text{FLOPs}^{WMSA} \approx 4hwC^2 + 2M^2hwC \quad (2)$$

Using the feature map dimensions from **Stage 2** ($h = H_2 \times W_2$, $c = C_2$) and the window size $M = 8$:

- Calculate the computational cost (in FLOPs) for the quadratic term $2(hw)^2C$ if Global MSA were used.
- Calculate the computational cost (in FLOPs) for the corresponding term $2M^2hwC$ using W-MSA.
- Calculate the **Speedup Factor** (Ratio: $\frac{\text{Global Cost}}{\text{Window Cost}}$) provided by the Swin architecture at this specific stage.

3 DETR Training: Forward Pass & Loss Calculation

Context: DETR frames object detection as a direct set prediction problem. A key component of its training process is the bipartite matching step, which forces a unique one-to-one assignment between the fixed set of model predictions and the ground truth objects. In this exercise, you will manually process the raw model outputs, solve the bipartite matching problem to find the optimal assignment, and calculate the final Hungarian loss to update the model.

Problem Setup: You are given a training sample containing **2 Ground Truth objects** and a DETR model with a fixed set size of $N = 4$ predictions.

(Note: For this exercise, all bounding boxes are defined solely by their center coordinates (x, y) ; width and height parameters are ignored for simplicity.)

1. Class Definitions: The dataset has 4 object classes (C_0 – C_3). The model predicts these plus a “No Object” class ($C_\emptyset$) for background:

- C_0 : **Eagle**
- C_1 : **Pigeon**
- C_2 : **Crow**
- C_3 : **Owl**
- $C_\emptyset$: **No Object**

2. Ground Truth (y): The sample image contains two objects:

- y_1 : Class = **Eagle** (C_0), Box Center = (2.0, 2.0)
- y_2 : Class = **Owl** (C_3), Box Center = (6.0, 6.0)

(Note: The Ground Truth set is effectively padded with two $\emptyset$ (i.e., “No Object” class) elements to reach size $N = 4$.)

3. Model Predictions ($\hat{y}$): The model outputs the following Logits (raw scores) and Box Center coordinates for the 4 predictions:

Prediction	Logits $[z_{eagle}, z_{pigeon}, z_{crow}, z_{owl}, z_{\emptyset}]$	Box Center (x, y)
$\hat{y}_1$	[0.0, 2.0, 0.0, 0.0, 0.5]	(2.0, 2.0)
$\hat{y}_2$	[2.0, 0.0, 0.0, 0.0, 0.5]	(3.0, 3.0)
$\hat{y}_3$	[0.0, 0.0, 0.0, 2.0, 0.5]	(6.0, 6.0)
$\hat{y}_4$	[0.0, 2.0, 0.0, 0.0, 0.5]	(10.0, 10.0)

Definitions & Formulas:

- **Class Probability:**

$$p_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (1)$$

- **Box Loss:**

$$L_{box}(b, \hat{b}) = \|b - \hat{b}\|_2 \quad (2)$$

- **Matching Cost:** Used to determine the optimal assignment $\hat{\sigma}$ by minimizing the total cost over all matches:

$$\hat{\sigma} = \underset{\sigma}{\operatorname{argmin}} \sum_{i=1}^N L_{match}(y_i, \hat{y}_{\sigma(i)}) \quad (3)$$

Where the pairwise cost is:

$$L_{match}(y_i, \hat{y}_{\sigma(i)}) = -\mathbb{1}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} L_{box}(b_i, \hat{b}_{\sigma(i)}) \quad (4)$$

- **Hungarian Loss:** Used for the final gradient update.

$$L_{Hungarian} = \sum_{i=1}^N \left[-\log(\hat{p}_{\hat{\sigma}(i)}(c_i)) + \mathbb{1}_{\{c_i \neq \emptyset\}} L_{box}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right] \quad (5)$$

Tasks:

- Preliminary Analysis (Qualitative):** Before performing any calculations, briefly inspect the "Logits" and "Box Center" values in the table above.
 - Which prediction seems to be a "hallucination" (high confidence for an object where none exists)?
 - Identify the prediction that is spatially accurate for the Eagle (y_1) but classifies it incorrectly.
 - Identify the prediction that classifies the Eagle correctly but is spatially inaccurate.
- Optimal Assignment (Bipartite Matching):** Determine the optimal one-to-one mapping $\hat{\sigma}$ between the ground truth set $\{y_1, y_2\}$ and the prediction set $\{\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4\}$ that minimizes the total matching cost defined above.
 - Calculate the required class probabilities and box distances.
 - Construct the cost matrix for the positive ground truth objects (2×4).
 - State clearly which prediction is assigned to y_1 (**Eagle**), which is assigned to y_2 (**Owl**), and which predictions are assigned to **No Object** ($\emptyset$).
- Total Training Loss:** Based on the optimal assignment derived in part (b), calculate the final scalar value of the **Hungarian Loss** ($L_{Hungarian}$) for this training step.

4 Point Transformer: Vector Attention Mechanism

The Point Transformer Layer utilizes a "Vector Attention" mechanism, distinct from the scalar dot-product attention found in standard Vision Transformers. The feature update is defined by:

$$y_i = \sum_{x_j \in \mathcal{X}(i)} \rho(\gamma(\varphi(x_i) - \psi(x_j) + \delta)) \odot (\alpha(x_j) + \delta) \quad (1)$$

- a) **Relation Modeling:** In standard attention, the interaction between a query and a key results in a scalar weight that represents similarity. Contrast this with the interaction term $\varphi(x_i) - \psi(x_j) + \delta$ in the equation above. How does the dimensionality of this interaction term differ from standard attention, and what does this allow the model to capture regarding the relationship between points i and j ?
- b) **Positional Encoding Injection:** In the equation, the positional encoding δ is injected at two distinct locations: once within the attention generation function ρ , and again in the value term $(\alpha(x_j) + \delta)$. Analyze the role of the second injection. Why might it be insufficient for 3D scene understanding to include positional information solely in the calculation of the attention weights?
- c) **Aggregation Dynamics:** The final aggregation utilizes the Hadamard product ($\odot$). Contrast the effect of this operation on the value vector $(\alpha(x_j) + \delta)$ against the standard scalar dot-product attention. How does this impact the information flow for different feature channels?

5 Coding Exercise on Transformers

In this week's coding exercise, you will train a vision transformer (ViT) for image classification on the CIFAR-10 dataset.

Tracking the performance of your training is a very important part of any Deep Learning project. For this reason, in this course we will use Weights & Biases to do so. If you don't have an account yet, please create one now, as you will need it for the coding exercise.

The provided instructions are for running the code in Google Colab.

1. Download and Decompress the Exercise Zip
 - Download the exercise zip `ADL_Ex04.zip`.
 - Decompress it to access its contents
2. Overview of the `ADL_Ex04` folder
 - `vision-transformer.ipynb`: Use this Jupyter notebook as an interactive terminal
 - `unit_tests.py`: Some basic and very simple tests to help make sure your implementation is correct.
 - `data` folder: Some validation data for the unit tests.
 - Code additions: Specifically look for the `### YOUR CODE HERE` and `### END CODE` in the notebook.
3. Upload the full folder to your Google drive and open the notebook in colab.
4. Change the runtime to GPU: Runtime → Change runtime type → T4 GPU. If the GPU runtime is not available, you can use the CPU runtime.
5. Mount your google drive as done in the first cell and make sure your `unit_tests.py` and `data` is accessible in the following cells. You may need to adjust paths if you uploaded the folder to some subdirectory.
6. Follow the instructions in the fourth cell (`!timeout 5m wandb init`) to link your Weights & Biases account.
7. After your Weights & Biases account is linked, you should comment (or delete) the same code cell (`!timeout 5m wandb init`), as otherwise it will wait for input every time you run an experiment.

8. Implement the missing parts.

Instructions for Code Implementation

You are required to implement specific functionalities within the classes and functions in the notebook. Make sure you pass all the unit tests in `unit_tests.py` before starting the training of the network.

1. Modifications for patch embeddings:

- *img_to_patch*: This function is responsible for converting input images into a sequence of image patches. You need to add code that will appropriately resize and reshape the input images into a flat array of patches.
- *PatchEmbedding*: In this class, implement the necessary transformations to convert image patches into flattened, embedded representations. This typically involves applying a linear transformation or using a small convolutional network.
- *PositionalEncoding*: This component is crucial for adding unique positional information to the patch embeddings, helping the model to retain information about the order and position of the image patches within the original image.

2. Modifications for the model:

- *ClassToken*: Implement functionality to prepend a class token to the sequence of embedded patches. This class token will ultimately hold the aggregate information used for classification after passing through the transformer encoder.
- *SelfAttention*: Code the self-attention mechanism within this class with multi-heads.
- *TransformerEncoder*: This class encapsulates multiple layers of the transformer encoder block, which includes self-attention and is responsible for encoding the sequence of image patches.
- *VisionTransformer*: As the main class integrating all components of the Vision Transformer. This includes invoking patch embedding, concatenating class_token, adding positional encodings, processing through several transformer encoder layers, and handling the output class token to produce a final classification result.

Once the setup is done, answer the following questions:

- By analyzing the graphs (train loss, test loss, test accuracy) of your first training, what do you notice?
- How does changing the patch size (try 8, and 4 in addition to the default 16) affect the test accuracy?

Now you should run a set of experiments, and draw some conclusions by analyzing the metric graphs. **IMPORTANT:** before running each experiment, change the experiment name in (`args['experiment_name']`), and restart the Colab session (Runtime → Restart session). In addition to the code, you will need to submit a document that includes, for each of the experiment, the graphs created during the experiment, and your conclusions. You can easily do so by using the “Reports” feature of Weights & Biases, which let you interchange graphs from your experiments and text. For example, for the experiment regarding learning rate, you should provide a graph with the comparison of the accuracy (or loss) for different learning rates, and your conclusions.

The experiments you need to run are the following (Optional):

- Change the learning rate. Try at least two lower and one higher rates.
- Change the depth of the network.
- Try different dropout rates.
- Feel free to experiment more (e.g., adding data augmentation). What is the best test accuracy you were able to achieve?